

Incident report

Summarised Cybersecurity Incident Report and Security Hardening Recommendations

1. Executive Summary

A controlled cybersecurity assessment was conducted against an **OWASP Juice Shop** environment to evaluate network visibility, web-application attack detection, and intrusion-detection capabilities.

The assessment involved **Nmap host and service discovery, XSS testing, Wireshark packet analysis, TCP traffic inspection, and Suricata IDS monitoring**. The testing was performed in a controlled laboratory environment consisting of a Windows system hosting Juice Shop and a Kali Linux virtual machine used for security monitoring.

The assessment demonstrated that HTTP traffic to the Juice Shop application could be captured and inspected using Wireshark. An HTTP request to the Juice Shop server on TCP port 3000 was successfully observed. The investigation also demonstrated how browser-generated requests associated with an XSS test can appear in network traffic.

Suricata was configured with local detection rules to identify suspicious XSS activity and other potentially malicious traffic. Testing also highlighted the importance of ensuring that the IDS is monitoring the correct network interface and traffic path.

Overall, the assessment demonstrated that **network monitoring, application security controls, IDS signatures, secure configuration, and proper segmentation must work together** to detect and prevent web-based attacks.

2. Scope of Assessment

The assessment covered:

- Host discovery using Nmap.
- Service and port identification.
- OWASP Juice Shop web-application testing.
- Cross-Site Scripting (XSS) testing using controlled payloads.
- HTTP traffic capture using Wireshark.
- TCP/HTTP stream inspection.
- Suricata IDS configuration and rule testing.

- Validation of network traffic visibility.
- Review of security weaknesses and recommended hardening measures.

The testing was performed against the intentionally vulnerable Juice Shop laboratory environment.

3. Environment

Component	Description
Target application	OWASP Juice Shop
Target operating system	Windows
Monitoring platform	Kali Linux
Virtualisation	VMware
Application protocol	HTTP
Application port	TCP/3000
Target IP observed during testing	192.168.18.81
Network analysis	Wireshark
IDS	Suricata
Network scanning	Nmap
Browser	Google Chrome

4. Incident / Assessment Timeline

Phase 1 — Host and Service Discovery

Nmap was used to identify the target host and determine which services and ports were accessible.

The assessment included service/version detection against the Juice Shop application, particularly TCP port 3000.

This established the network location and service exposure of the application.

Phase 2 — Web Application Testing

The Juice Shop application was accessed through the browser and controlled XSS testing was performed.

Example test payloads included harmless proof-of-concept inputs such as:

```
<img src=x onerror=alert(XSS)>
```

and:

```
<a href="javascript:alert(1)">test</a>
```

The purpose was to determine whether suspicious input could be submitted to the application and whether the resulting traffic could be detected at the network level.

Phase 3 — Suricata Detection

Suricata was configured to monitor Juice Shop traffic and detect suspicious XSS-related patterns.

A local Suricata rule was created for the testing environment.

Phase 4 — Network Traffic Verification

Tcpdump and Wireshark were used to determine whether the traffic generated by the Windows Juice Shop environment was reaching the Kali monitoring system.

Traffic associated with:

192.168.18.81:3000

Was successfully observed.

This confirmed that the monitoring interface could see Juice Shop traffic.

Phase 5 — Wireshark Analysis

Wireshark captured an HTTP request similar to:

```
GET /x HTTP/1.1  
Host: 192.168.18.81:3000
```

The server responded with:

```
HTTP/1.1 304 Not Modified
```

The /x request was investigated in relation to the XSS test.

The analysis determined that this request alone did **not constitute definitive proof that the XSS payload itself had been transmitted**. It demonstrated HTTP communication and could potentially be a browser-generated request resulting from processing the test input.

TCP Stream analysis was therefore used to provide additional context.

5. Key Findings

Finding 1 — Web Application Exposure

The Juice Shop application was accessible over the network using HTTP on TCP port 3000.

Risk

An unnecessarily exposed web application can increase the attack surface of a production environment.

Severity

Medium, depending on network exposure.

Finding 2 — XSS Testing

Controlled XSS testing demonstrated that potentially malicious HTML/JavaScript input could be submitted to the intentionally vulnerable Juice Shop application.

Risk

In a real application, successful XSS could allow malicious JavaScript to execute in another user's browser, potentially resulting in:

- Session compromise.
- Credential theft.
- Malicious actions performed in the victim's session.
- Website defacement.
- Phishing.
- Data theft.

Severity

High if exploitable in a production application.

Finding 3 — Unencrypted HTTP Traffic

The Juice Shop traffic was observed using HTTP rather than HTTPS.

Risk

HTTP traffic can potentially expose:

- URLs.
- Parameters.
- HTTP headers.
- Application data.
- Cookies.
- Authentication information.

Network attackers capable of intercepting the traffic could potentially obtain sensitive information.

Severity

High for production systems transmitting authentication or sensitive data.

Finding 4 — Authentication Token Exposure

The captured HTTP request contained an authentication token in the Cookie header.

This information was visible during packet inspection.

Risk

If an authentication token were intercepted by an unauthorized party, it could potentially be used to impersonate the associated session.

Severity

Critical/High, depending on token privileges and server-side controls.

Recommendation: Authentication tokens must be protected using HTTPS and appropriate cookie security controls.

6. Root Cause Summary

The weaknesses identified during the assessment can broadly be attributed to:

1. Use of an intentionally vulnerable web application.
2. Insufficient web-application input validation/output encoding.
3. Use of unencrypted HTTP.
4. Exposure of authentication information within observable network traffic.
5. IDS rule-management/configuration errors.

6. Lack of automated IDS configuration validation.
 7. Potential network-monitoring visibility gaps.
 8. Insufficient separation between application and monitoring traffic.
 9. Lack of layered security controls.
-

7. Security Hardening Recommendations

7.1 Implement HTTPS

Production applications should not transmit authentication credentials, session cookies, or sensitive application data over plain HTTP.

Implement:

- TLS 1.2 or preferably TLS 1.3.
- Valid certificates.
- HTTP-to-HTTPS redirection.
- HSTS.
- Strong TLS configuration.
- Removal of insecure HTTP endpoints where possible.

This prevents network observers from directly reading HTTP application content.

7.2 Implement Strong XSS Protection

Applications should treat user-supplied input as untrusted.

Recommended controls include:

Input validation

Validate expected input types and reject unexpected HTML/JavaScript where it is not required.

Context-aware output encoding

Encode user-controlled data before placing it into:

- HTML.
- HTML attributes.
- JavaScript.
- CSS.

- URLs.

Content Security Policy

Deploy a strong **Content-Security-Policy (CSP)** to reduce the impact of injected JavaScript.

Avoid unsafe DOM operations

Developers should avoid unsafe patterns such as directly inserting untrusted input through mechanisms equivalent to `innerHTML`.

Prefer safe DOM APIs and trusted templating frameworks.

7.3 Secure Authentication Cookies

Authentication cookies should use:

Secure
Http Only
Same Site

where appropriate.

Secure

Prevents transmission over unencrypted HTTP.

Http Only

Helps prevent JavaScript from directly reading the cookie.

Same Site

Helps reduce certain cross-site request risks.

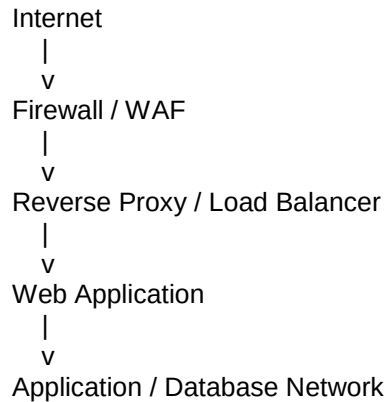
Session tokens should also have:

- Short appropriate lifetimes.
 - Server-side invalidation.
 - Rotation where appropriate.
 - Strong signing/encryption mechanisms.
 - Least-privilege session permissions.
-

8. Network Segmentation

Production web applications should not be directly exposed to every network segment.

A recommended architecture is:



Monitoring infrastructure should receive appropriate traffic through:

- Network TAPs.
- SPAN/mirror ports.
- Properly configured virtual networking.
- Cloud traffic mirroring where applicable.

This ensures that security monitoring systems can observe relevant traffic without unnecessarily exposing internal systems.

9. Web Application Firewall

Deploy a **Web Application Firewall (WAF)** in front of production web applications.

A WAF can provide an additional layer of protection against:

- XSS.
- SQL injection.
- Malicious HTTP requests.
- Path traversal.
- Automated attacks.
- Suspicious request patterns.

A WAF should complement—not replace—secure application development.

10. Suricata Hardening

Suricata should be configured as part of a layered detection strategy.

Recommended practices include:

Validate rules before deployment

Run rule/configuration validation before restarting Suricata.

This helps identify:

- Duplicate signatures.
- Invalid syntax.
- Unsupported keywords.
- Configuration errors.

Maintain unique SIDs

Every custom Suricata rule should have a unique signature ID.

For example:

```
sid:1000001;  
sid:1000002;  
sid:1000003;
```

Do not reuse the same SID for multiple unrelated rules.

Monitor Suricata health

Security teams should monitor:

- Service status.
- Rule-loading errors.
- fast.log.
- eve.json.
- Dropped packets.
- Capture statistics.

An IDS that silently stops processing traffic creates a significant detection gap.

11. Improve IDS Detection

XSS detection should not rely on one string such as:

<script

Attackers can modify payloads through encoding, case changes, alternative HTML elements, or JavaScript techniques.

Detection should therefore use multiple indicators and, where possible, dedicated web-application security controls.

Suricata rules should be tested against:

- Normal traffic.
- Known malicious patterns.
- Encoded variants.
- False-positive scenarios.

Detection rules should also be reviewed regularly.

12. Protect Sensitive Logs and PCAP Files

Wireshark captures and IDS logs may contain sensitive information such as:

- Authentication cookies.
- Session tokens.
- Usernames.
- URLs.
- Internal IP addresses.
- Application data.

Access to packet captures and security logs should therefore be restricted.

Recommended controls include:

- Role-based access.
- Encryption at rest.
- Restricted file permissions.
- Appropriate retention periods.
- Secure deletion.
- Redaction before reports are distributed.

13. Secure Development Lifecycle

XSS and similar vulnerabilities should ideally be identified before applications reach production.

Implement:

1. Secure coding standards.
 2. Developer security training.
 3. Static Application Security Testing (SAST).
 4. Dynamic Application Security Testing (DAST).
 5. Dependency scanning.
 6. Code review.
 7. Penetration testing.
 8. Automated security testing in CI/CD.
 9. Vulnerability remediation tracking.
-

14. Recommended Incident Response Process

If similar activity is detected against a production application, the following process should be followed:

Detect

Identify the suspicious request through IDS, WAF, application logs, or SIEM.

Validate

Determine whether the activity is legitimate testing, a false positive, or an actual attack.

Contain

If necessary:

- Block the source.
- Disable the affected endpoint.
- Revoke compromised sessions.
- Apply temporary WAF rules.

Investigate

Review:

- HTTP logs.
- IDS alerts.

- Firewall logs.
- Authentication logs.
- PCAP data.
- Application logs.

Eradicate

Remove the vulnerability and address the underlying cause.

Recover

Restore normal operation and monitor for recurrence.

15. Risk Prioritisation

| Finding | Risk | Priority |
|-------------------------------|-------------|----------|
| XSS vulnerability | High | Critical |
| HTTP instead of HTTPS | High | Critical |
| Monitoring visibility gaps | High | High |
| Web application exposure | Medium | Medium |
| Insufficient layered controls | Medium/High | High |

16. Overall Security Assessment

The assessment demonstrated that the laboratory environment could be successfully monitored at the network level and that tools such as Nmap, Wireshark, and Suricata can complement one another during a security investigation.

However, the testing also highlighted several important security principles:

- Application security:**
 Vulnerabilities such as XSS must be prevented within the application rather than relying solely on network detection.
- Encryption:**
 Sensitive HTTP traffic must be protected using HTTPS.
- Monitoring:**
 IDS systems must have visibility into the traffic they are expected to monitor.
- Configuration management:**
 Security tools must be properly configured and their rules validated before deployment.

5. **Layered security:** WAF, secure coding, HTTPS, authentication controls, network segmentation, IDS, logging, and SIEM monitoring should work together.

17. Conclusion

The controlled assessment successfully demonstrated the relationship between **web-application security testing and network-level detection**.

Nmap provided information about the target's network and service exposure. XSS testing demonstrated how malicious input can be introduced into a vulnerable web application. Wireshark provided visibility into the resulting HTTP communication, while Suricata provided an opportunity to detect suspicious traffic through IDS signatures.

For a production environment, the most important improvements are to **eliminate XSS through secure coding and output encoding, enforce HTTPS, protect authentication tokens, deploy appropriate WAF controls, properly configure and validate Suricata, maintain network visibility, segment networks, and continuously monitor security events**.

The assessment demonstrates the importance of a **defence-in-depth approach**, where prevention, detection, monitoring, response, and secure development operate together rather than relying on a single security control.